



chapter 4

■ ■ ■ ■ ■ ■ ■ ■ ■

串

主讲人：周瑜

4.1

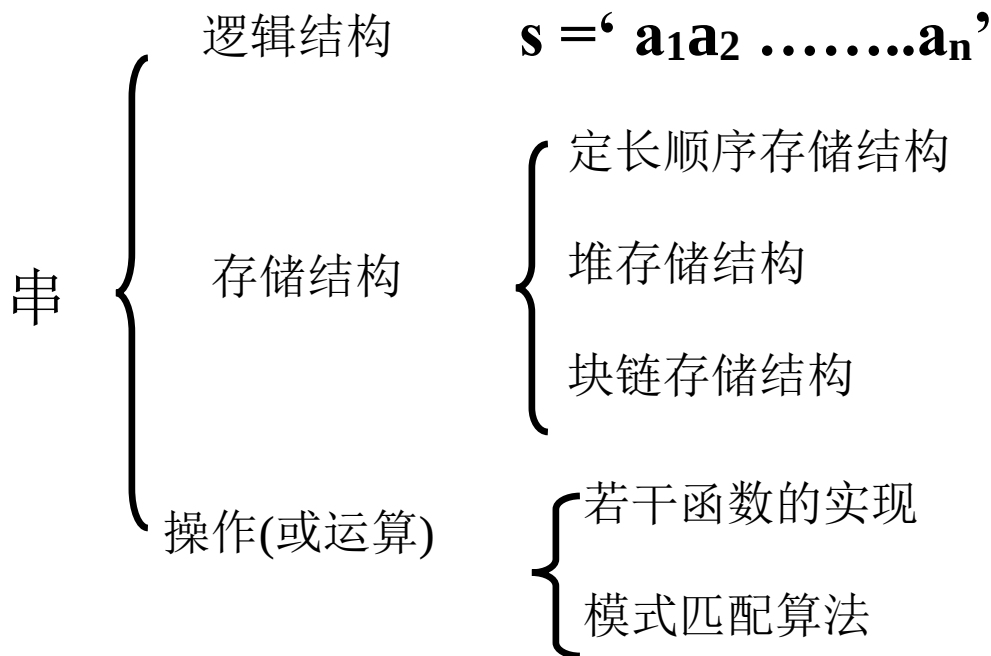
串类型的定义

4.2

串的实现和表示

4.3

串的模式匹配算法

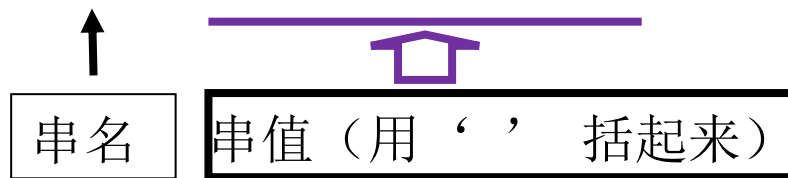


模式匹配即子串定位运算，即如何实现 $\text{Index}(S, T, \text{pos})$ 函数

精华：KMP算法——快速（用 $\text{next}[j]$ 或 $\text{nextval}[j]$ ）

串即字符串，是由零个或多个字符组成的有限序列，是数据元素为单个字符的特殊线性表。

记为： $s = 'a_1 a_2 \dots a_n'$ ($n \geq 0$)



隐含结束符 ‘\0’，
即ASCII码NUL

为何要单独讨论“串”类型？

- 1) 字符串操作比其他数据类型更复杂（如拷贝、连接操作）
- 2) 程序设计中，处理对象很多都是串类型。

若干术语：

串长：串中字符的个数（ $n \geq 0$ ）。 $n=0$ 时称为空串 $\emptyset$ 。

空格（白）串：由一个或多个空格符组成的串。

空串和空格串有无区别？

答：有区别。

空串 (Null String) 是指长度为零的串；

而空格串 (Blank String), 是指包含一个或多个空格字符
' ' (空格键) 的字符串。

子串： 串S中任意个连续的字符序列叫S的子串; S叫主串。

子串位置： 子串的第一个字符在主串中的序号。

字符位置： 字符在串中的序号。

串相等： 串长度相等，且对应位置上字符相等。

例1：现有以下4个字符串：

a = 'BEI' b = 'JING' c = 'BEIJING' d = 'BEI JING'

问：① 他们各自的长度？ **a = 3, b = 4, c = 7, d = 8**

② a是哪个串的子串？在主串中的位置是多少？

a是c和d的子串，在c和d中的位置都是1

③ 空串是哪个串的子串？a是不是自己的子串？

“空串是任意串的子串；任意串S都是S本身的子串，除S本身外，S的其他子串称为S的真子串。”

——《数据结构与算法》中山大学出版社

串的抽象数据类型定义

ADT String{

Objects: $D = \{a_i \mid a_i \in \text{CharacterSet}, i=1, 2, \dots, n, n \geq 0\}$

Relations: $R1 = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i=2, \dots, n \}$

functions:

//至少有13种基本操作

StrAssign(&T, chars) // 串赋值, 生成值为chars的串T

StrCompare(S, T) // 串比较, 若 $S > T$, 返回值大于0...

StrLength(S) // 求串长, 即返回串S中的元素个数

Concat(&T, S1, S2) // 串连接, 用T返回 $S1 + S2$ 的新串

SubString(&Sub, S, pos, len) // 求S中pos起长度为len的子串

.....

int Index(S, T, pos) // 子串定位函数 (模式匹配), 返回位置

Replace(&S, T, V) // 用子串V替换子串T

}ADT String

C语言中已有类似的串运算函数!

最小操作子集

复习：C语言中常用的串运算

注：用C处理字符串时，要调用标准库函数 `#include<string.h>`

C

串比较： `int strcmp(char *s1,char *s2);`

求串长： `int strlen(char *s);`

串连接： `char strcat(char *to,char *from)`

子串T定位： `char strchr(char *s,char *c);`

.....

类C

`StrCompare(S,T)`

`StrLength(S)`

`Concat(&T, S1, S2)`

`Index(S, T, pos)`

注：Concat操作=concatenation，把多个短字符串合并为长字符串

4.1

串类型的定义

例1: 设 $s = 'I \square AM \square A \square STUDENT'$, $t = 'GOOD'$, $q = 'WORKER'$, 求:

$StrLength(s) = 14$

$StrLength(t) = 4$

$SubString(\&sub, s, 8, 7) = 'STUDENT'$

$SubString(\&sub, t, 2, 1) = 'O'$

$Index(s, 'A') = 3$

$Index(s, t) = 0$ (s 中没有 $t = 'GOOD'$)

$Replace(\&s, 'STUDENT', q) = 'I \square AM \square A \square WORKER'$

当 $s = 'I \square A \square M \square A \square STUDENT'$ 时,

$INDEX(s, 'A', pos) = 3$, 若想搜索后面那个 'A' 怎么办?

答: $INDEX(s, 'A')$ 返回的只是“第一次”出现的位置。

如果还要搜索后面的 A, 则 pos 变量应当跟着变。

也就是说, 要把得到的“第一次”位置再代入

$INDEX(s, 'A', pos)$ 函数中循环操作才行。

例2： 设 $s = 'I \square AM \square A \square STUDENT'$, $t = 'GOOD \square'$, 求：

$\text{Concat}(\text{SubString}(s, 6, 2), \text{Concat}(t, \text{SubString}(s, 8, 7))) = ?$

解： 因为

$\text{SubString}(s, 6, 2) = 'A \square';$

$\text{SubString}(s, 8, 7) = 'STUDENT'$

$\text{Concat}(t, \text{SubString}(s, 8, 7)) = 'GOOD \square STUDENT'$

所以：

$\text{Concat}(\text{SubString}(s, 6, 2), \text{Concat}(t, \text{SubString}(s, 8, 7)))$

$= 'A \square GOOD \square STUDENT'$

4.1

串类型的定义

4.2

串的实现和表示

4.3

串的模式匹配算法

串与线性表的运算有所不同，是以“串的整体”作为操作对象。

例如查找某子串，在主串某位置上插入一个子串等。

串的三种机内表示方法：

定长顺序存储表示（静态存储）

用一组地址连续的存储单元存储串值的字符序列，属静态存储方式。

堆分配存储表示（静态存储）

用一组地址连续的存储单元存储串值的字符序列,但存储空间是在程序执行过程中动态分配而得。

串的块链存储表示（链式存储）

链式方式存储

定长顺序存储特点：

用一组连续的存储单元来存放串，直接使用定长的字符数组来定义，数组的上界预先给出，故称为静态存储分配。

例如：

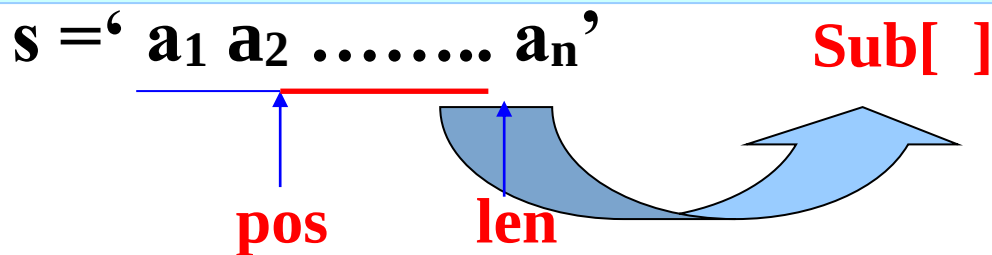
```
#define Maxstrlen 255  //用户可用的最大串长
typedef unsigned char SString[ Maxstrlen + 1 ] SString;
SString s;  //s 是一个可容纳255个字符的顺序串。
```

说明：

- 有些实现用SString[0]来存放串长信息（如pascal语言）；
- C语言约定在串尾加结束符 ‘\0’，以利操作加速，但不计入串长（用首址和串长、或首址和尾标记来描述串数组）
- 若字符串超过Maxstrlen 则自动截断（静态数组存不进去）。

例：用顺序存储方式编写求子串函数SubString(&Sub,S,pos,len)

将串S中从第pos个字符开始、长度为len的字符序列复制到串Sub中。
(注：考虑到函数的通用性，应当让串Sub的预留长度与S一样)



```
Status SubString(SString &sub, SString S, int pos, int len )
```

```
{ if(pos<1 || pos>S[0] || len<0 || len>S[0]-pos+1) return ERROR;
```

//若pos和len参数越界，则告警

```
Sub[1.....len]=S[pos.....pos+len-1];
```

```
Sub[0]=len; return OK;
```

```
}
```

讨论：想存放超长字符串怎么办？改用动态分配的一维数组——堆

堆分配存储特点：仍用一组连续的存储单元来存放串，但存储空间是在程序执行过程中动态分配而得。

堆T的存储结构描述：

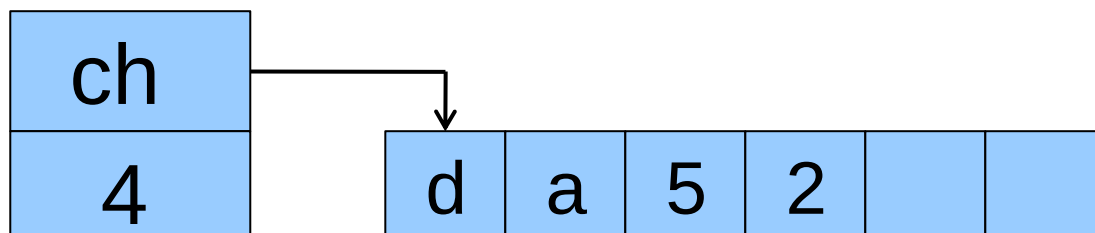
```
Typedef struct {  
    char *ch; //若非空串,按串长分配空间; 否则ch=NULL  
    int length; //串长度  
}HString
```

各分量书写方式：

T.ch

T.length

堆分配存储示例：串 ‘da52’ 的堆分配存储结构如下图所示：



思路：利用malloc函数合理预设串长空间。

特点：若在操作中串值改变，还可以利用realloc函数按新串长度增加空间（即动态数组概念）。

例1：编写建堆函数

```
Status StrAssign(HString &T, char *chars)
{ //生成一个串T, T值←串常量chars
  if (T.ch) free(T.ch); //释放T原有空间
  for (i=0, c=chars; c; ++i, ++c); //求chars的串长度i
  if ( !i ) {T.ch = NULL; T.length = 0;}
  else{
    if (!(T.ch = (char*)malloc( i *sizeof(char))))
      exit(OVERFLOW);
    T.ch[0..i-1] = chars[0..i-1];
    T.length =i;
  }
  Return OK;
} //StrAssign
```

C是指针变量，可以自增！
意即每次后移一个数据单元。

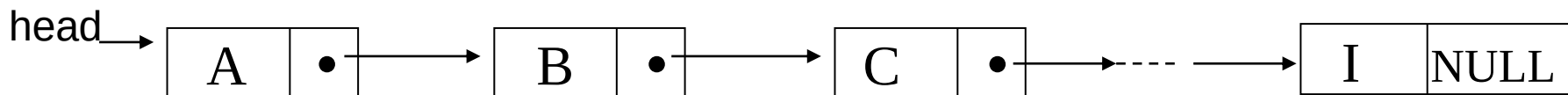
直到终值为“假”停止，串尾特征是
 $c = '\backslash 0' = \text{NULL} = 0$

例2：用“堆”方式编写串插入函数

```
Status StrInsert ( HString &S, int pos, HString T )
{ //在串S的第pos个字符之前 (包括尾部) 插入串T
  if (pos<1||pos>S.length+1) return ERROR; //pos不合法则告警
  if(T.length){ //只要串T不空, 就需要重新分配S空间, 以便插入T
    if (! ( S.ch=(char*)realloc(S.ch, (S.length+T.length)*sizeof(char)) )) exit(OVERFLOW);
    //若开不了新空间, 则退出
    for ( i=S.length-1; i>=pos-1; --i ) S.ch[i+T.length] = S.ch[i];
    // 为插入T而腾出pos之后的位置, 即从S的pos位置起全部字符均后移
    S.ch[pos-1..pos+T.length-2] = T.ch[0..T.length-1]; //插入T, 略/0
    S.length += T.length; //刷新S串长度
  } return OK;
} //StrInsert
```

链式存储特点：用链表存储串值，易插入和删除。

法1：链表结点的数据分量长度取1（个字符）



法2：链表结点（数据域）大小取n（例如n=4）



讨论：法1存储密度为 $\frac{1}{2}$ ；法2存储密度为 $\frac{9}{15} = \frac{3}{5}$

若数据元素很多，用法2存储更优—称为块链结构

块链类型定义：

```
typedef struct {           //定义用链式存储的串类型
    Chunk *head;           //头指针
    Chunk *tail;           //尾指针
    int curLen;             //结点个数
} Lstring;                //串类型只用一次，前面可以不加结构名称
```

对块链表的整体描述

```
#define CHUNKSIZE 80      //每块大小，可由用户定义
typedef struct Chunk {     //首先定义结点类型
    char ch [ CHUNKSIZE ]; //每个结点中的数据域
    struct Chunk * next;    //每个结点中的指针域
}Chunk;
```

对每个结点的描述

4.1

串类型的定义

4.2

串的实现和表示

4.3

串的模式匹配算法

算法目的：确定主串中所含子串第一次出现的位置（定位）

定位问题称为串的模式匹配，典型函数为 $\text{Index}(S, T, \text{pos})$

算法种类：

BF算法（又称古典的、经典的、朴素的、穷举的）：带回溯，速度慢

KMP算法：避免回溯，匹配速度快

BF算法的实现

例1: $S = \text{'ababcabcacbab'}$, $T = \text{'abcac'}$, $pos = 1$,

求: 串T在串S中第pos个字符之后的位置。

BF算法设计思想:

- 将主串S的第pos个字符和模式T的第1个字符比较,
若相等, 继续逐个比较后续字符;
若不等, 从主串S的下一字符 ($pos+1$) 起, 重新与T
第一个字符比较。

BF算法的实现

例1: $S = \text{'ababcabcacbab'}$, $T = \text{'abcac'}$, $pos = 1$,

求: 串T在串S中第pos个字符之后的位置。

直到主串S的一个连续子串字符序列与模式T相等。返回值为S中与T匹配的子序列第一个字符的序号, 即匹配成功。否则, 匹配失败, 返回值 0。

BF算法的实现

例如，设目标串 $s = \text{"cddcdc"}$ ，模式串 $t = \text{"cdc"}$ 。 s 的长度为 $n(n=6)$ ， t 的长度为 $m(m=3)$ 。

用指针 i 指示目标串 s 的当前比较字符位置，用指针 j 指示模式串 t 的当前比较字符位置。BF模式匹配过程如下所示。

BF算法的实现

第 1 次匹配	$s = cddcdc$ $\begin{array}{c} \\ t = cdc \end{array}$	$i = 3$ $j = 3$	失败
第 2 次匹配	$s = cddcdc$ $\begin{array}{c} \\ t = cdc \end{array}$	$i = 2$ $j = 1$	失败
第 3 次匹配	$s = cddcdc$ $\begin{array}{c} \\ t = cdc \end{array}$	$i = 3$ $j = 1$	失败
第 4 次匹配	$s = cddcdc$ $\begin{array}{c} \\ t = cdc \end{array}$	$i = 6$ $j = 3$	成功

$$i = i - j + 2;$$

$$j = 1;$$

BF算法的实现

求子串位置的定位函数Index(S, T, pos) 的BF算法:

```
Int Index_BP(SString S, SString T, int pos) //见教材P79
{ //返回子串T在主串S中第pos个字符之后的位置。若不存在, 则函数值为0.其中, T非空,
  1≤pos≤StringLength(S)

  i=pos;   j=1;
  while ( i<=S[0] && j<=T[0] ) {
    //如果i,j二指针在正常长度范围,
    if (S[i] == T[j] ) {++i, ++j; } //则继续比较后
    else {i=i-j+2; j=1;} //若不相等, 指针后退重新匹配
  }
  if(j>T[0]) return i-T[0]; //T子串指针j正常到尾, 说明匹配成功,
  //否则属于i>S[0]情况, i先到尾就不正常
} //Index_BP
```

i-j=回到起始点之前,
i-j+1=回到首字符,
i-j+2=定位到第二个字符

else return 0;

BF算法的时间复杂度

主串长 n ；子串长 m 。可能匹配成功的位置 ($1 \sim n-m+1$)。

① 最好的情况下：

第 i 个位置匹配成功，前 $i-1$ 个位置每个位置只比较了一次，
第 i 个位置比较了 ($i-1+m$) 次，平均比较次数：

$$\sum_{i=1}^{n-m+1} p_i(i-1+m) = \frac{1}{n-m+1} \sum_{i=1}^{n-m+1} (i-1+m) = \frac{1}{2}(m+n)$$

最好情况下算法的平均时间复杂度 $O(n+m)$ 。

BF算法的时间复杂度

主串长 n ，子串长 m 。可能匹配成功的位置($1 \sim n-m+1$)。

②最坏的情况下：

第 i 个位置匹配成功，但每个 $i-1$ 位置都比较了 m 次，总共比较了($i*m$)次，平均比较次数：

$$\sum_{i=1}^{n-m+1} p_i (i \times m) = \frac{m}{n-m+1} \sum_{i=1}^{n-m+1} i = \frac{1}{2} m(n-m+2)$$

设 $n \gg m$ ，最坏情况下的平均时间复杂度为 $O(n*m)$ 。

分析算法的思想：

从主串S的第pos个字符起和模式的第一个字符比较，若相等，继续比较后续字符；否则，从主串的下一个字符起再重新和模式的字符比较。以此类推，直到模式T中的每个字符依次和主串S中的一个连续的字符序列相等，则匹配成功，函数返回T首字符在S中的位置；否则匹配不成功。

分析算法的思想：

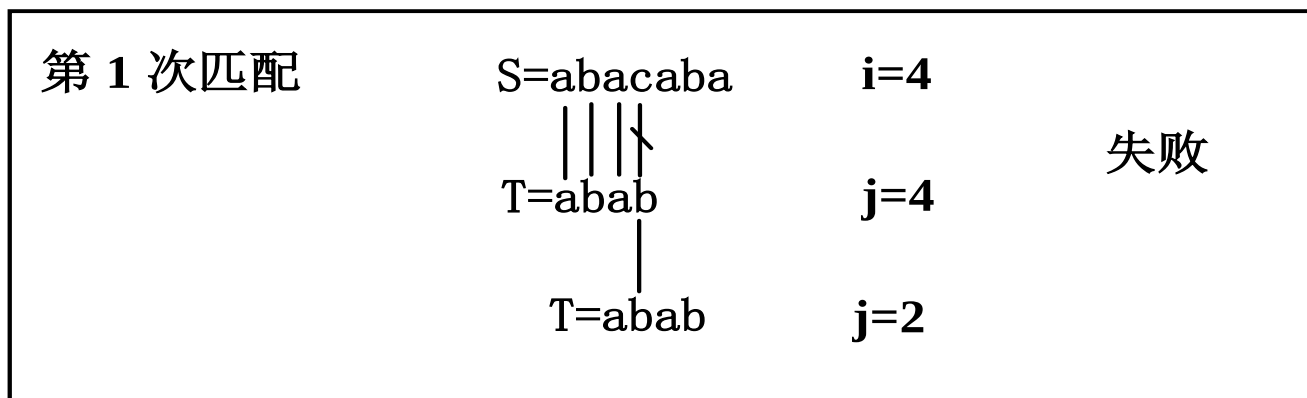
问题：当主串与模式串存在多次部分匹配时，就显得效率低下。如0、1串在 $T=“00001”$, $S=“000000000000001”$ 时，每次都有四次的多余比较，而0、1串又是普遍存在的。

KMP算法（特点：速度快）

- ① KMP算法设计思想
- ② KMP算法的推导过程
- ③ KMP算法的实现（关键技术:计算next[j]）
- ④ KMP算法的时间复杂度

模式匹配的改进算法-KMP算法

KMP算法是D.E.Knuth、J.H.Morris和V.R.Pratt共同提出的，该算法消除了主串指针的回溯，提高算法效率。



因 $t_1 \neq t_2, s_2 = t_2$, 必有 $s_2 \neq t_1$, 又因 $t_1 = t_3, s_3 = t_3$, 所以必有 $s_3 = t_1$ 。

因此,第二次匹配可直接从 $i=4, j=2$ 开始。

① KMP算法设计思想

尽量利用已经部分匹配的结果信息，尽量让*i*不要回溯，加快模式串的滑动速度，例：

i
 $\downarrow$
 $S = \text{'a b a b c a b c a c b a b'}$
 $T = \text{'a b c a c'}$
 $\uparrow$
 k

i
 $\downarrow$
 $S = \text{'a b a b c a b c a c b a b'}$
 $T = \text{'a b c a c'}$
 $\uparrow$
 k

i i i
 $\downarrow$ $\downarrow$ $\downarrow$
 $S = \text{'a b a b c a b c a c b a b'}$
 $T = \text{'a b c a c'}$
 $\uparrow$
 k

① KMP算法设计思想

两个问题：

- ① 如何由当前部分匹配结果确定模式向右滑动的新比较起点k？
- ② 模式应该向右滑多远才是高效率的？

② KMP算法的推导过程

(1)

$S = \text{'a b a b c a b c a c b a b'}$
 $T = \text{'a b c a c'}$

设目前打算与T的第k字符开始比较，k是追求的新起点

则T的1~k-1位 = S前i-(k-1)~i-1位,即(4-2)式含义

$\text{'T}_1 \dots \text{T}_{k-1}'$

② KMP算法的推导过程

(2)

$S = \text{'a b a b c a b c a c b a b'}$
 $T = \text{'a b c a c'}$

刚才肯定是在S的*i*处和T的第*j*字符处失配

则T的*j-(k-1) ~ j-1*位 = S前*i-(k-1) ~ i-1*位，即(4-3) 式含义，‘ $T_{j-(k-1)} \dots T_{j-1}$ ’ 截取一段，但*k*有限制， $1 < k < j$

② KMP算法的推导过程

两式联立可得： $'T_1 \dots T_{k-1}' = 'T_{j-(k-1)} \dots T_{j-1}'$

加速的前提：T首与 T_j 处有相同子串！

注意：j为当前已知的失配位置，我们的目标是计算新起点k。
式中仅剩一个未知数k，理论上已可解！

新起点k怎么求？

根据模式串T的规律： $'T_1...T_{k-1}' = 'T_{j-(k-1)}...T_{j-1}'$

由当前失配位置j(已知)，可以归纳出计算新起点k的表达式。

令 $k = \text{next}[j]$ (k与j显然具有函数关系)，则

$$\text{next}[j] = \begin{cases} 0 & \text{当 } j=1 \text{ 时} & // \text{不比较} \\ \max \{ k \mid 1 < k < j \text{ 且 } 'T_1...T_{k-1}' = 'T_{j-(k-1)}...T_{j-1}' \} & \\ 1 & \text{其他情况} \end{cases}$$

取T首与 T_j 处最大的相同子串

讨论：

(1) $\text{next}[j]$ 有何物理意义？

(2) $\text{next}[j]$ 具体怎么求？——即KMP算法的实现

(1) $\text{next}[j]$ 有何物理意义?

$T = \text{'a b a a b c a c'}$

$$\text{next}[j] = \max \{ k \mid 1 < k < j \text{ 且 } 'T_1 \dots T_{k-1}' = 'T_{j-(k-1)} \dots T_{j-1}' \}$$

模式串从第1位往右
直到k-1位

模式串从j的前一位往
左经过k-1位

$\text{next}[j]$ 函数表征着模式T中最大相同前缀子串和后缀子串（真子串）的长度。

可见，模式中相似部分越多，则 $\text{next}[j]$ 函数越大，它既表示模式T字符之间的相关度越高，也表示j位置以前与主串部分匹配的字符数越多。

(2) $\text{next}[j]$ 具体怎么求？

计算 $\text{Next}[j]$ 的方法：

- 当 $j=1$ 时， $\text{Next}[j]=0$ ；// $\text{Next}[j]=0$ 表示根本不进行字符比较
- 当 $j>1$ 时， $\text{Next}[j]$ 的值为：模式串的位置从1到 $j-1$ 构成的串中所出现的首尾相同的子串的最大长度加1。

无首尾相同的子串时 $\text{Next}[j]$ 的值为1。

// $\text{Next}[j]=1$ 表示从模式串头部开始进行字符比较

怎样计算模式T所有可能的失配点 j 所对应的 $\text{next}[j]$ ？

4.3

串的模式匹配算法

例： 模式串 T: a b a a b c a c

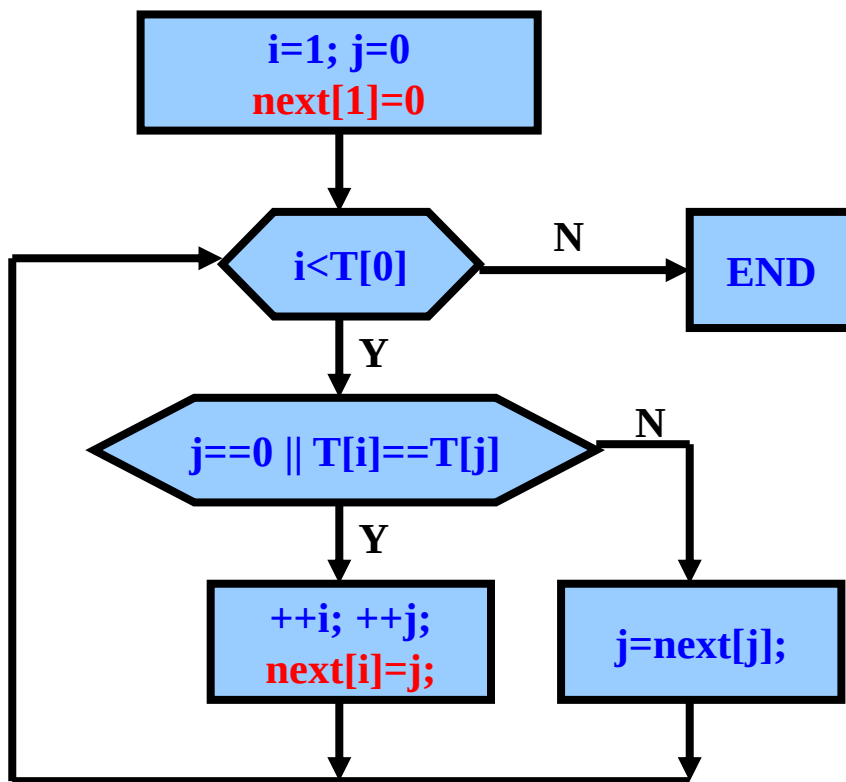
可能失配位 j: 1 2 3 4 5 6 7 8

新匹配位 $k = \text{next}[j]$: 0 1 1 2 2 3 1 2

next[j]与s无关，可以预先计算

刚才已归纳：
$$\text{next}[j] = \begin{cases} 0 & \text{当 } j = 1 \text{ 时} \\ \max \{ k \mid 1 < k < j \text{ 且 } 'T_1 \dots T_{k-1}' = 'T_{j-(k-1)} \dots T_{j-1}' \} & \text{其他情况} \\ 1 & \end{cases}$$

求解next[j]流程图（递推）



如何用递推方式求出最大相同子串的长度？换言之，如何让电脑替我们求出最大相同子串呢？

```
void get_next(SString T, int &next[ ] ){  
    //求模式串T的next函数值并存入数组next[ ]  
    i=1; j=0; next[1]=0;  
    while(i<T[0] ){ //T[0]为T串之长度  
        if(j==0||T[i]==T[j])  
            {++i; ++j; next[i]=j;} //首次必j=0  
        else j=next[j];  
    }  
} // get_next
```

补充：递推与递归的区别：

递推：由“小”到“大”递进

递归：由“大”到“小”嵌套

例如：求 $f(n)=n!$

① 递推法：

```
fact=1;
for ( i=1; i<=n; i++)
    fact*= i;
```

② 递归法：

```
long int fact(n)
    int n;
    {   long f;
        if(n>1)f=n*fact(n-1);
        else f=1;
        return(f); }
```

③ KMP算法的实现—即Index()操作的实现

第一步，先把模式T所有可能的失配点j所对应的`next[j]`计算出来；

第二步：执行定位函数Index_kmp （与BF算法模块非常相似）

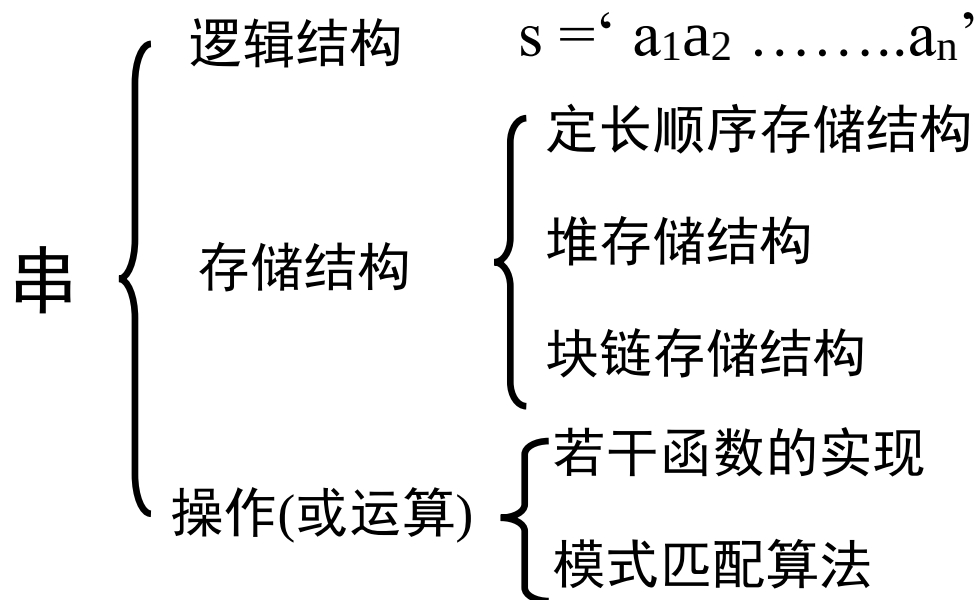
```
Int Index_KMP(SString S, SString T, int pos) {  
    i=pos;    j=1;  
    while ( i<=S[0] && j<=T[0] ) {  
        if (j==0|| S[i] == T[j] ) {++i, ++j} //不失配则继续比较后续字符  
        else {j=next[j];} //特点：S的i指针不回溯，而且从T的k位置开始匹配  
    }  
    if(j>T[0]) return i-T[0]; //子串结束，说明匹配成功  
    else return 0;  
} //Index_KMP
```

④ KMP算法的时间复杂度

回顾BF的最恶劣情况：S与T之间存在大量的部分匹配，
比较总次数为： $(n-m+1)*m=O(n*m)$

而KMP的情况是：由于指针*i*无须回溯，比较次数仅为*n*，
即使加上计算next[j]时所用的比较次数*m*，比较总次数也
仅为*n+m=O(n+m)*，大大快于BF算法。

注意：由于BF算法在一般情况下的时间复杂度也近似于 $O(n+m)$ ，
所以至今仍被广泛采用。



模式匹配即子串定位运算，即如何实现 $\text{Index}(S, T, \text{pos})$ 函数

BF算法——古典

KMP算法——快速（用 $\text{next}[j]$ ）